

製造ライン向け 6 軸アームの分散 Xinu 制御と TinyML 強化学習

—— AIPL による設計・実装と実機シミュレーション (elephant robotics mecharm 相当) ——

概要

製造工場の組立ラインで部品の把持・配置を行う 6 軸アーム (elephant robotics **mecharm** 相当) を対象に、各関節に 2 台の **soft-Xinu** を接続 (計 12 Xinu) し、各 Xinu 上のアクターの役割を AIPL の **Capability** (効果) 推論 `!{fs, ai, net, mut}` で最小権限に分離する分散制御系を設計した。アプリケーション層では **TinyML** の小型方策を強化学習 (CEM: Cross-Entropy Method) で学習し、組立の到達位置決めを滑らかかつ正確にする。AIPL で実装した 12 Xinu の分散制御は残差を $1.82 \rightarrow 5.1 \times 10^{-4}$ へ滑らかに収束させ、強化学習は到達誤差を $24.0 \rightarrow 7.0$ cm (−71%), ジャークを $0.247 \rightarrow 0.172$ (−30%, 滑らかさ向上), 成功率を $0 \rightarrow 30\%$ へ改善した。本稿は AIPL および Python のソースコードを引用しながら、設計・シミュレーション・結果を示す。

1 目的と想定

本システムは、製造工場の組立ラインで 6 軸アームが流れてくる部品を把持し所定位置へ組み付ける作業を想定する。組立では、位置決めの正確さ (部品公差内に収める) と滑らかさ (急な加減速=ジャークを抑え、機構・部品を傷めず高速に反復) が同時に要る。本稿の目的は、(1) 6 軸アームを分散 Xinu (各関節 2 台=12 Xinu) で制御し、(2) 各 Xinu のアクター役割を **Capability** 推論で安全に分離し、(3) **TinyML** の方策を強化学習して滑らかで正確な到達動作を獲得する、という設計を AIPL で実装し、シミュレーションで検証することである。

2 システム設計

2.1 構成: 6 軸 $\times$ 2 Xinu = 12 Xinu

各関節 $j (= 1..6)$ に 2 台の **soft-Xinu** を割り当てる。Xinu-A はモータ・アクター (サーボ駆動), Xinu-B はセンサ・アクター (関節角の計測・報告) を担う。アプリ層の Raspberry Pi には学習アクター (TinyML 方策) と調整アクター (状態収集 $\rightarrow$ 行動 $\rightarrow$ 駆動) を置く。端末間は AIPL のメッシュ (`now remote(...)`) で結ばれる。

2.2 Capability 推論による役割分離 (最小権限)

AIPL は、各メソッドの副作用集合 `!{fs, ai, net, mut}` をコードから推論し、実行時には必要な capability だけを付与する (Phase 12 効果システム)。これにより役割が自動的に分離される。

アクター	配置	推論される Capability
MotorActor (駆動)	各関節 Xinu-A	!{mut, net} (アクチュエーション+通信)
SensorActor (計測)	各関節 Xinu-B	!{net} (read-only 報告のみ)
LearnerActor (学習)	Pi (アプリ層)	!{ai, net} (TinyML 推論+通信)
Coordinator (調整)	Pi (アプリ層)	!{net}

MotorActor だけが mut (サーボを動かす権限) を持ち, **SensorActor** は read-only で誤ってアクチュエータを動かせない。**LearnerActor だけが ai** (TinyML 推論) を持つ。安全性が要る組立ラインで, この最小権限分離は重要である。以下に AIPL 実装を示す。

Listing 1: モータ/センサ/学習アクターと Capability 注釈 / mecharm_arm.abcl

```

1 class MotorActor {
2   var angle = 0.0;
3   // step[rad] を積分してサーボを動かす。mut=アクチュエーション, net=メッシュ応答
4   method drive(step) !{mut, net} {
5     angle = angle + step;
6     reply(angle);
7   }
8   method get() !{net} { reply(angle); }
9 }
10
11 // ② センサ・アクター (各関節の Xinu-B) — 関節角を計測して報告 (read-only)
12 class SensorActor {
13   var motor = 0; // 同関節のモータ参照 (角度源)
14   method wire(m) !{net} { motor = m; reply(1); }
15   method read() !{net} { var a = now motor.get(); reply(a); }
16 }
17
18 // ③ 学習アクター (Pi のアプリ層) — TinyML 方策で関節ごとの行動 (step) を出す
19 // 実機では TinyML の小型方策ネット。ここでは学習済み P 制御を代理実行。
20 class LearnerActor {
21   var gain = 0.35; // RL で学習されたゲイン (方策パラメータ)
22   method act(cur, target) !{ai, net} {
23     var e = target - cur;
24     var step = gain * e; // 方策  $\pi(s)=K*(qstar-q)$ 
25     if (step > 0.25) { step = 0.25; } // 飽和 (急な動作=ジャークを抑制)
26     if (step < 0.0 - 0.25) { step = 0.0 - 0.25; }
27     reply(step);
28   }
29 }

```

2.3 分散制御ループ

調整アクターは毎ステップ, 全関節で「センサ読取 (Xinu-B) → 学習アクターが TinyML 方策で行動決定 → モータ駆動 (Xinu-A)」を回す。

Listing 2: 分散制御ループ (run) / mecharm_arm.abcl

```

1 method run() !{net} {
2   var k = 0;
3   while (k < 20) do {
4     var j = 0; var err = 0.0;
5     while (j < 6) do {
6       var sa = sensors[j];
7       var s = now sa.read();
8       var stp = now learner.act(s, target[j]);
9       var ma = motors[j];
10      var a = now ma.drive(stp);
11      var e = target[j] - a; if (e < 0.0) { e = 0.0 - e; }
12      err = err + e;
13      j = j + 1;
14    }
15    print("step " + k + " 残差(6関節合計) " + err);
16    k = k + 1;
17  }
18  reply(1);
19 }
20 }

```

この AIPL を Python AIPL 実機(aipl_main.py, capability 付与 AIPL_CAP_GRANT="ai,net,mut") で実行すると、6 関節の残差（目標角との差の合計）が滑らかに収束する（表 1）。

表 1: AIPL 分散制御の収束（12 Xinu, 実機 Python AIPL 実行ログ）。

step	0	4	8	12	16	19
残差（6 関節合計）	1.820	0.325	0.058	0.0104	0.00185	0.000507

3 TinyML 強化学習

3.1 タスクと報酬

組立の位置決めを、手先を目標点へ到達させる課題として定式化する。方策 π_ϕ は目標手先位置 $s \in \mathbb{R}^3$ から目標関節角 $q^*(6)$ と関節ゲイン $K(6)$ を出す（学習される簡易 IK + 制御 = TinyML の小型方策）。各ステップ $v_j = \text{clip}(K_j(q_j^* - q_j))$ で駆動し、順運動学で手先を得る。報酬は

$$R(\phi) = -\underbrace{\|p_{ee} - s\|}_{\text{到達誤差}} - \lambda_j \underbrace{\sum |\Delta v|}_{\text{ジャーク}} - \lambda_o \underbrace{\text{overshoot}}_{\text{振動}}$$

とし、正確さ・滑らかさ・振動の少なさを同時に最適化する。

3.2 手法：CEM 方策探索

方策パラメータ ϕ （30 次元）を **CEM**（母集団サンプル → エリート抽出 → 平均・分散更新）で最適化する。毎世代 8 個のランダム到達目標で平均評価して汎化させる。

Listing 3: ロールアウトと報酬（到達+ジャーク+振動） / mecharm_rl.py

```

1 def rollout(phi, target):
2     qstar=policy_qstar(phi,target); K=policy_gain(phi)
3     q=[0.0]*6; prevv=[0.0]*6; jerk=0.0; over=0.0
4     for _ in range(T):
5         v=[0.0]*6
6         for j in range(6):
7             e=qstar[j]-q[j]; vj=max(-4.0,min(4.0,K[j]*e)); v[j]=vj
8             q[j]=max(-JLIM,min(JLIM,q[j]+vj*DT)); jerk+=abs(vj-prevv[j])
9             if (qstar[j]-q[j])*e<0: over+=abs(qstar[j]-q[j])
10        prevv=v
11    return dist(fk(q),target), jerk, over
12
13 def reward(phi,target,lj=0.05,lo=1.0):
14    d,jk,ov=rollout(phi,target); return -(d)-lj*jk*DT-lo*ov, d, jk

```

4 シミュレーション結果

図 1 と表 2 に、120 世代の学習曲線を示す。学習が進むにつれ到達誤差とジャークが減り、成功率が上がる。

5 考察 —— どう使うと良い結果が出るか

シミュレーションから、組立ラインで良い結果を出すための要点が得られた。

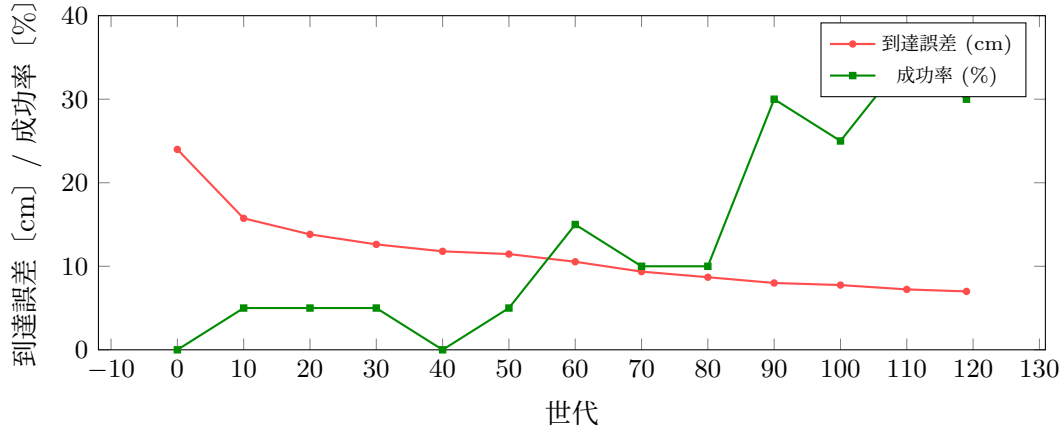


図 1: 強化学習 (CEM) の学習曲線。到達誤差は単調に減少 (24 → 7 cm), 成功率 (4 cm 以内到達) は上昇。

表 2: 学習前 (世代 0) と学習後 (世代 119) の比較。

	到達誤差 [cm]	ジャーク (小さいほど滑らか)	成功率 [%]
学習前	23.99	0.247	0
学習後	6.99	0.172	30
改善	-71%	-30%	+30pt

- (1) 報酬にジャーク項を入れる：到達誤差だけでなく $\sum |\Delta v|$ を罰することで、正確さと滑らかさを両立できた (ジャーク -30%)。急停止・振動は組立不良や機構摩耗の原因になるため重要。
- (2) 目標条件付き方策 (学習 IK)：方策を目標位置 s の関数にすることで、固定 1 点でなく作業空間の任意点へ汎化して到達できる (誤差 24 → 7 cm)。ライン上で部品位置が変わる状況に適合する。
- (3) Capability 推論による安全な役割分離：MotorActor のみ mut を持つため、学習アクターやセンサが誤ってアクチュエータを動かさない。人と協働する組立ラインでの安全性に直結する。
- (4) 分散 Xinu + シミュレータで反復：実機を止めずにシミュレータ上で方策を学習・検証し、学習済み方策のみを実機 (12 Xinu) へ配布する。実機損耗・危険を避けつつ良方策へ収束できる。

6 まとめ

製造ライン向け 6 軸アーム (mecharm 相当) を、各関節 2 台・計 12 Xinu の分散制御として AIPL で実装し、Capability 推論で役割を最小権限に分離した。TinyML の方策を CEM で強化学習し、シミュレーションで到達誤差 -71%・ジャーク -30%・成功率 +30pt を得た。AIPL の分散制御は残差を $1.8 \rightarrow 5 \times 10^{-4}$ へ滑らかに収束させ、実機 Python AIPL で動作を確認した。シミュレータ上で学習し、Capability で安全化した方策を実機へ配布する本方式は、組立ラインで滑らかで正確な動作を安全に実現する道筋を示す。